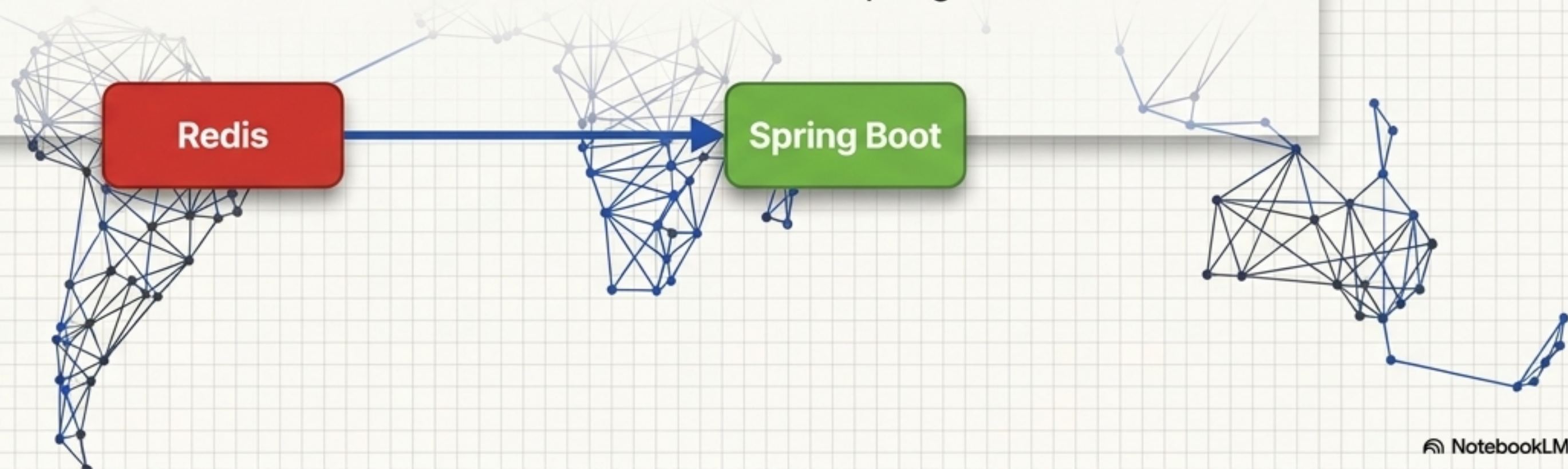


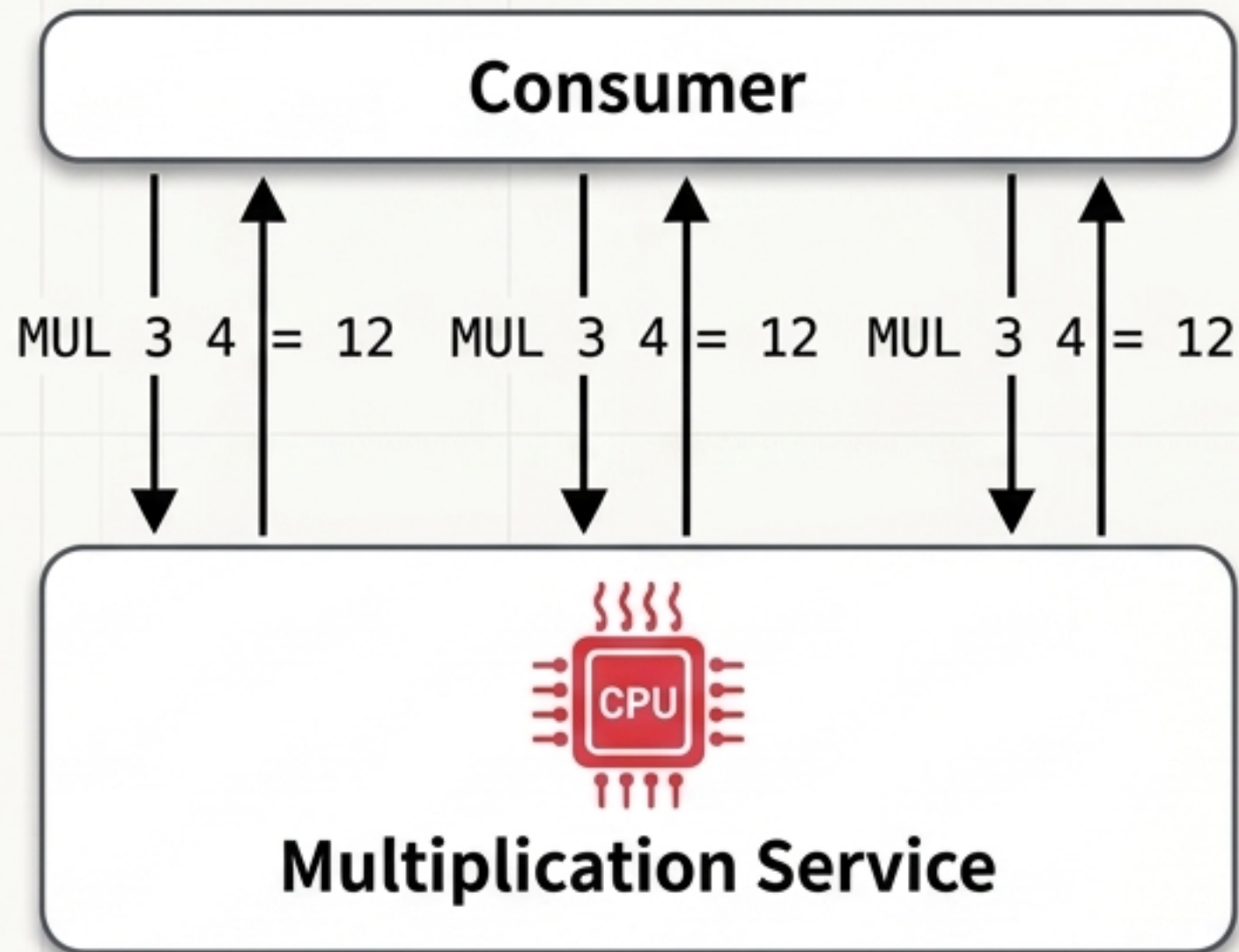
고속 데이터 하이웨이와 현대적 블루프린트 엔터프라이즈 캐싱과 Redis의 완성

대규모 애플리케이션의 지연 시간 극복을 위한 아키텍처 패턴부터 Spring Boot 실무 구현까지



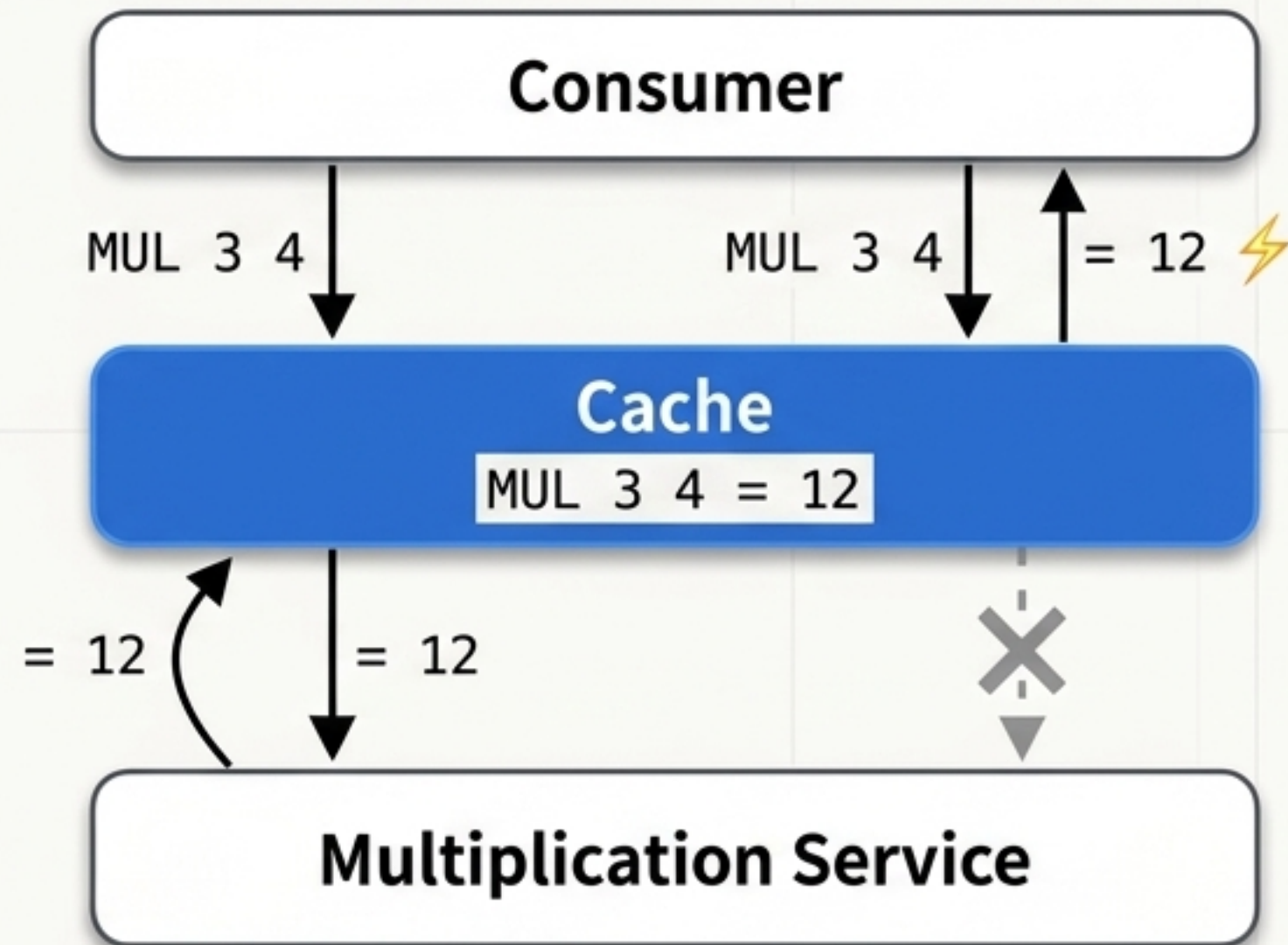
캐싱의 본질: 중복된 비용의 제거

캐시가 없을 때 (No Cache)



서비스는 자신이 이미 동일한 연산을 수행했다는 사실을 알지 못합니다.
3번의 요청은 3번의 리소스 소모를 의미합니다.

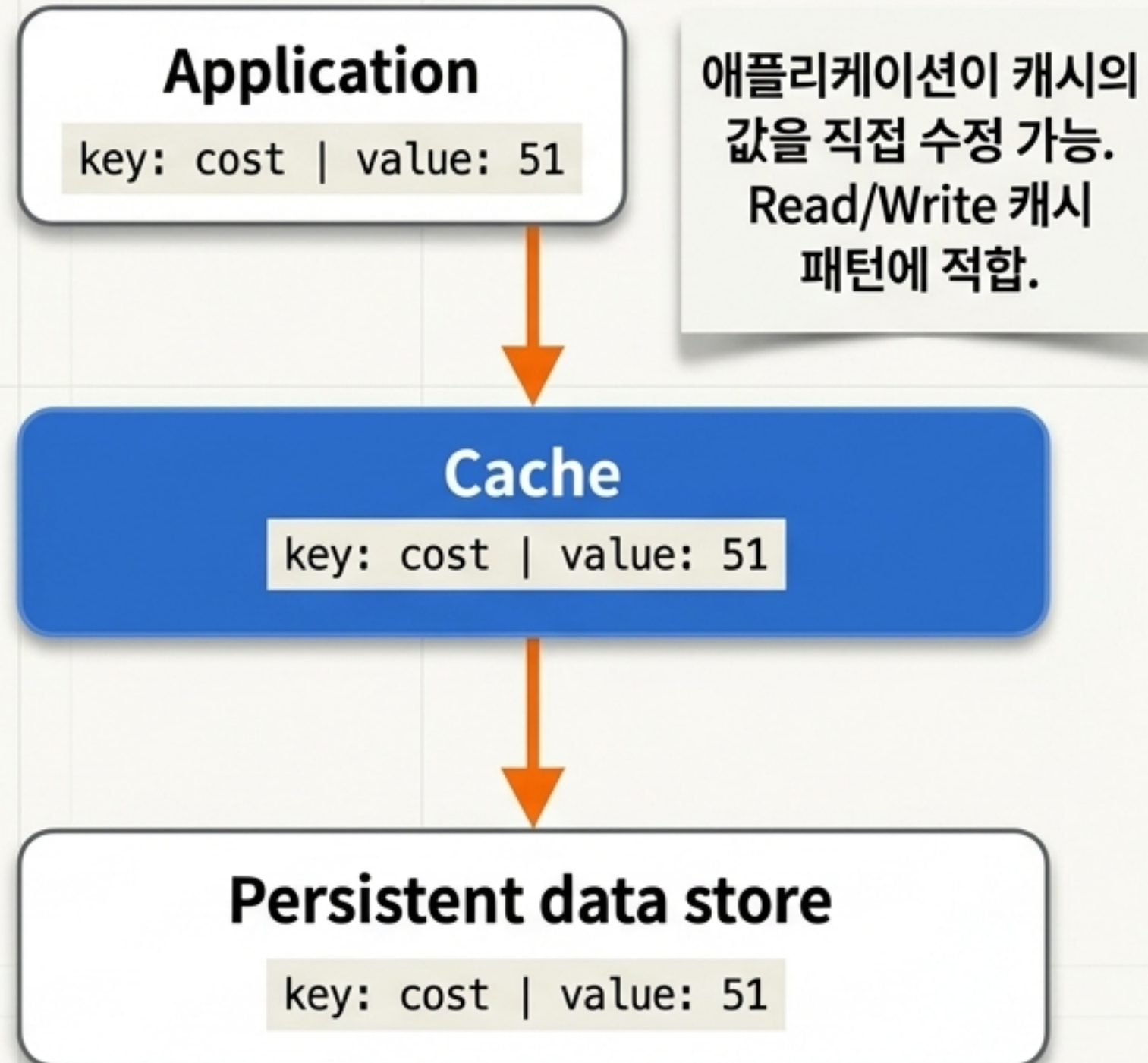
캐시가 있을 때 (With Cache)



단 한 번의 연산.
이후의 요청은 연산 없이 캐시에서 즉시 반환되어 지연 시간(Latency)을 비약적으로 단축합니다.

데이터 플로우: 동적(Dynamic) vs 정적(Static) 캐시

Dynamic Cache



Static Cache



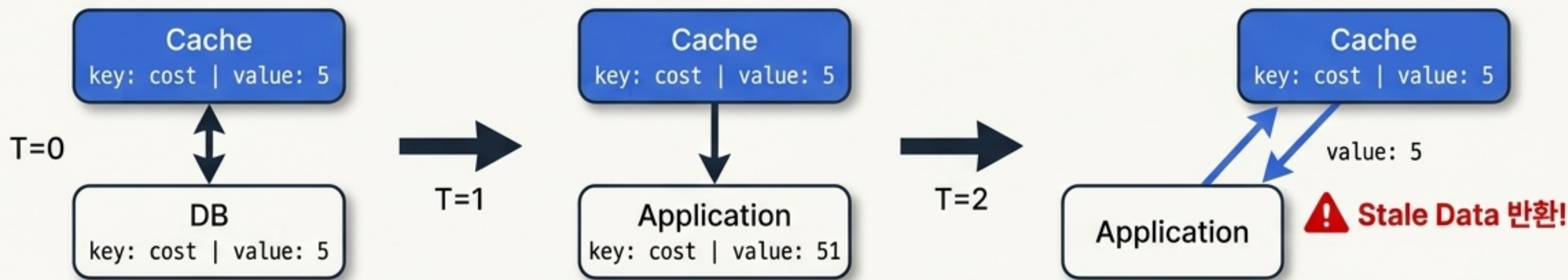
캐싱의 두 가지 치명적 함정

위험 1: 사이드 이펙트(Side Effects)의 누락



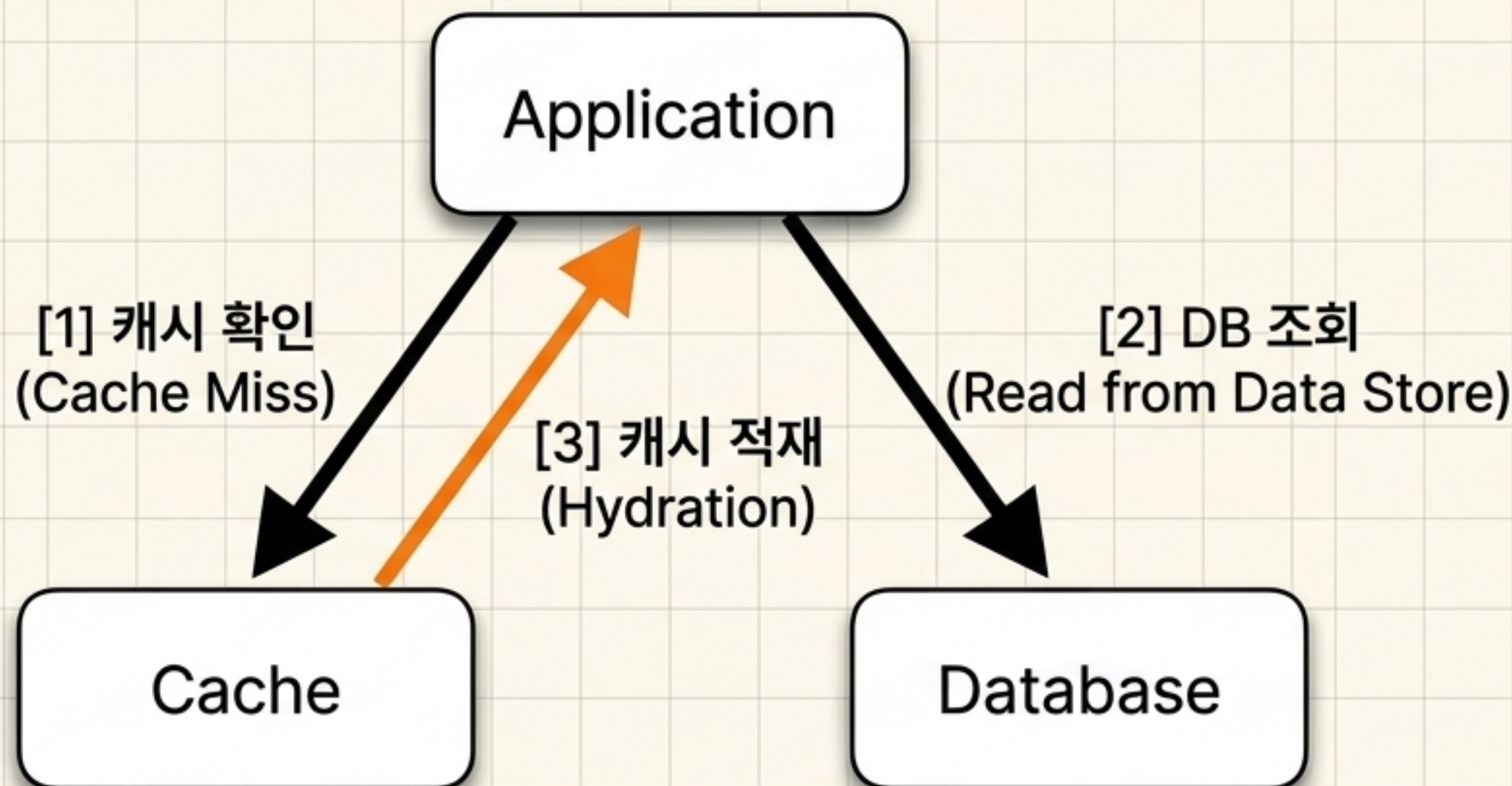
캐시가 응답을 대신함으로써, 백엔드 서비스가 수행해야 할 필수적인 외부 상태 변경(결제 승인, 사용자 로그 기록 등)이 누락될 위험성.

위험 2: 캐시 불일치 (Cache Inconsistency)



원본 데이터는 변경되었으나 캐시가 동기화되거나 무효화되지 않아 구버전의 데이터를 반환하는 상태.

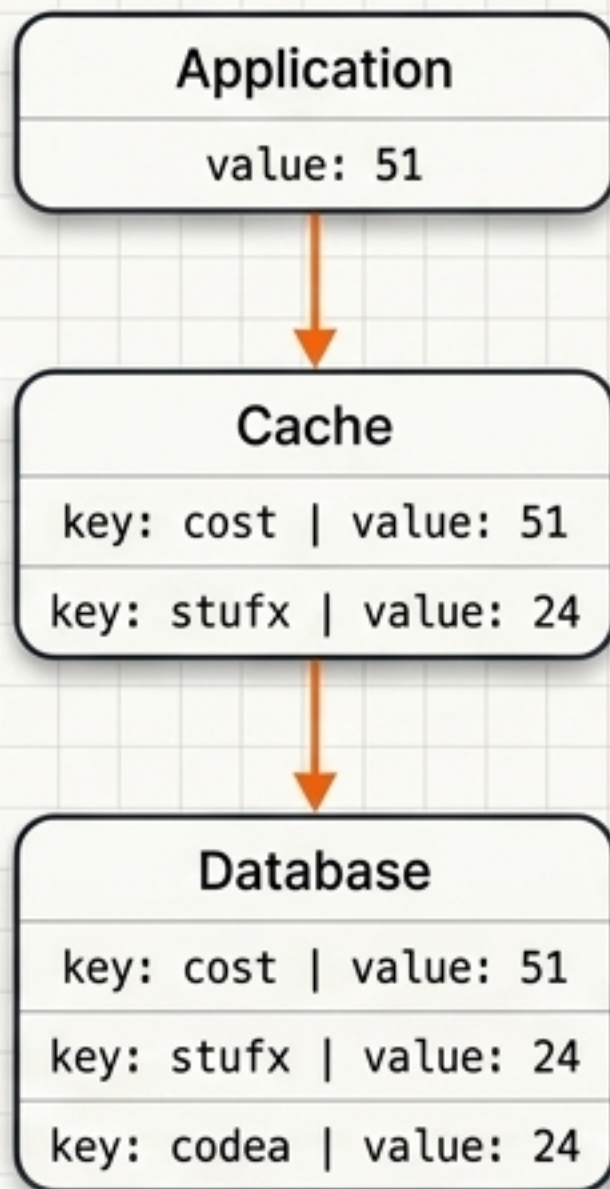
아키텍처 패턴 1: Cache-Aside (Lazy Loading)



캐시와 데이터베이스는 철저히 독립적입니다.
데이터 정합성 유지의 모든 책임은 전적으로 애플리케이션(Application)에 있습니다.

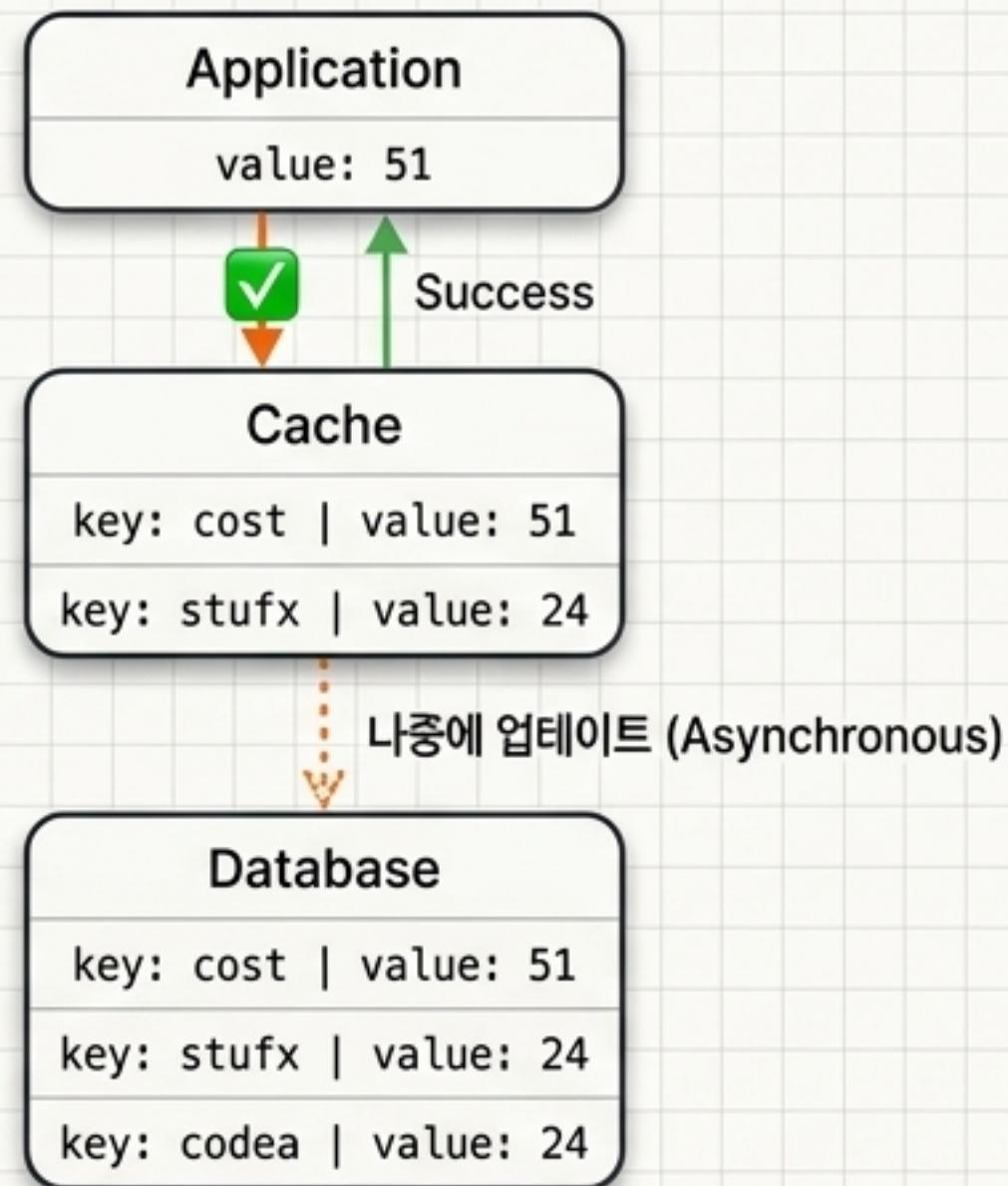
아키텍처 패턴 2: Write-Through vs Write-Behind

Write-Through



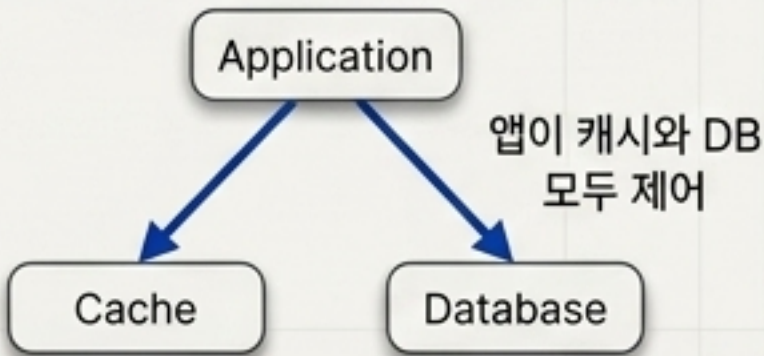
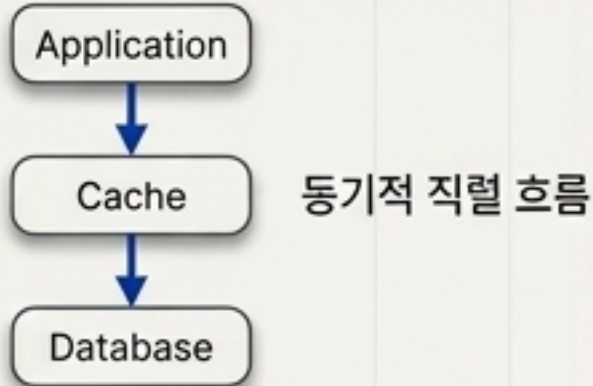
캐시를 먼저 업데이트하고, 캐시가 DB를 동기적으로 업데이트.
강한 일관성(Strong Consistency)을 보장하지만 쓰기 지연(Write Latency)이 발생.

Write-Behind / Write-Back



캐시에만 쓰고 즉시 반환. 백그라운드에서 비동기적으로 DB에 쓰기 수행.
빠른 쓰기 성능을 얻지만, 시스템 장애 시 데이터 유실(Data Loss) 리스크 존재.

의사 결정을 위한 캐시 쓰기 전략 거부매트릭스

전략 (Strategy)	데이터 흐름 (Data Flow)	강점 (Pros)	리스크 (Risks)
Cache-Aside	 <pre>graph TD; App[Application] --> Cache[Cache]; App --> DB[Database];</pre>	캐시 장애가 DB 접근에 치명적 영향을 주지 않음	초기 접속 시 캐시 미스로 인한 지연 (Cold Start)
Write-Through	 <pre>graph TD; App[Application] --> Cache[Cache]; Cache --> DB[Database];</pre>	완벽한 데이터 일관성 보장	데이터 쓰기 시 두 번의 왕복으로 인한 성능 저하
Write-Behind	 <pre>graph TD; App[Application] --> Cache[Cache]; Cache -. DB[Database];</pre>	극대화된 쓰기 성능 및 DB 부하 감소	동기화 전 캐시 노드 다운 시 최신 데이터 증발

메모리 한계의 극복: 퇴출(Eviction) 알고리즘 매트릭스

Redis의 메모리가 가득 찼을 때(maxmemory), 새 데이터를 위한 공간을 확보하는 전략(maxmemory-policy).



LRU (Least-Recently Used)

마지막으로 접근한 '시간' 기준. 가장 오랫동안 사용되지 않은 데이터 퇴출.

명령어: `allkeys-lru`



LFU (Least-Frequently Used)

접근한 '횟수' 기준. 가장 적게 조회된 데이터 퇴출.

명령어: `allkeys-lfu`



Oldest Stored (FIFO)

캐시에 들어온 순서 기준. 가장 먼저 적재된 데이터 퇴출.

Redis 기본 동작 외 별도 구성 필요

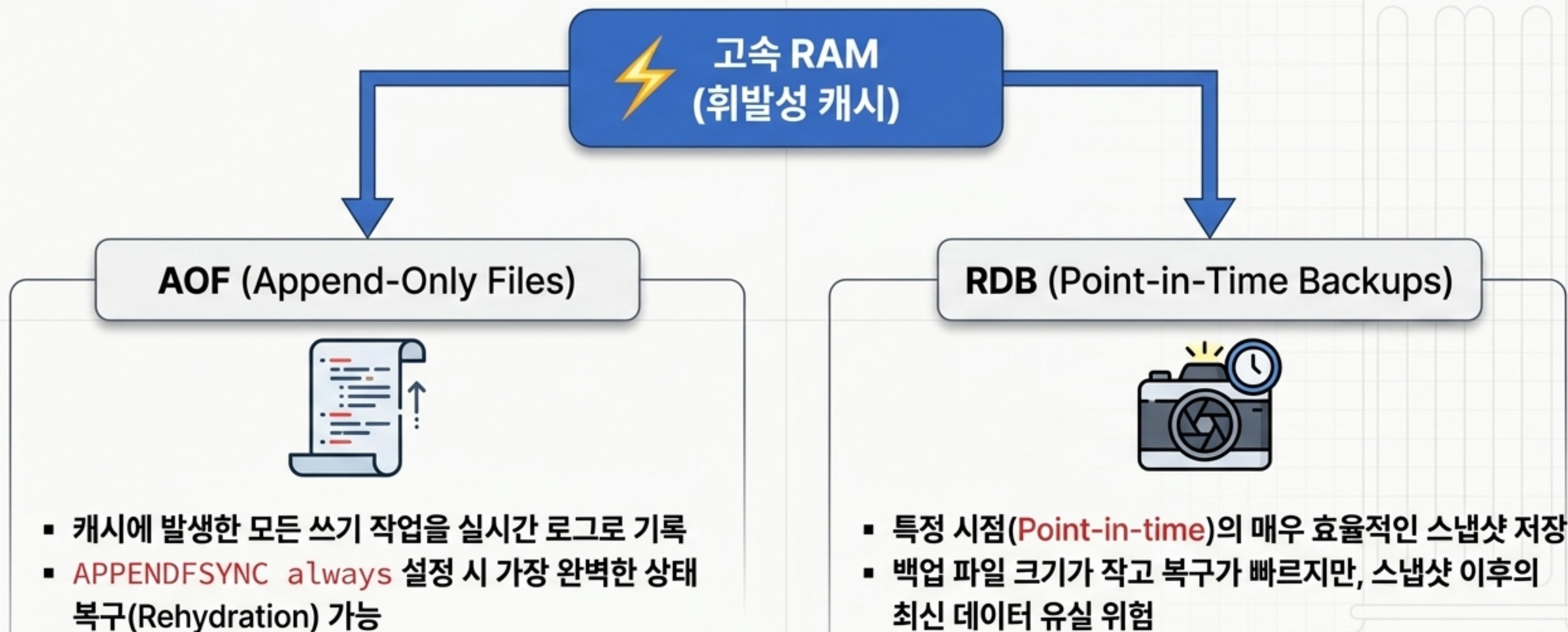


TTL (Time-to-Live)

지정된 수명이 다하면 공간 부족 여부와 상관없이 자동 퇴출(예: 세션 관리용).

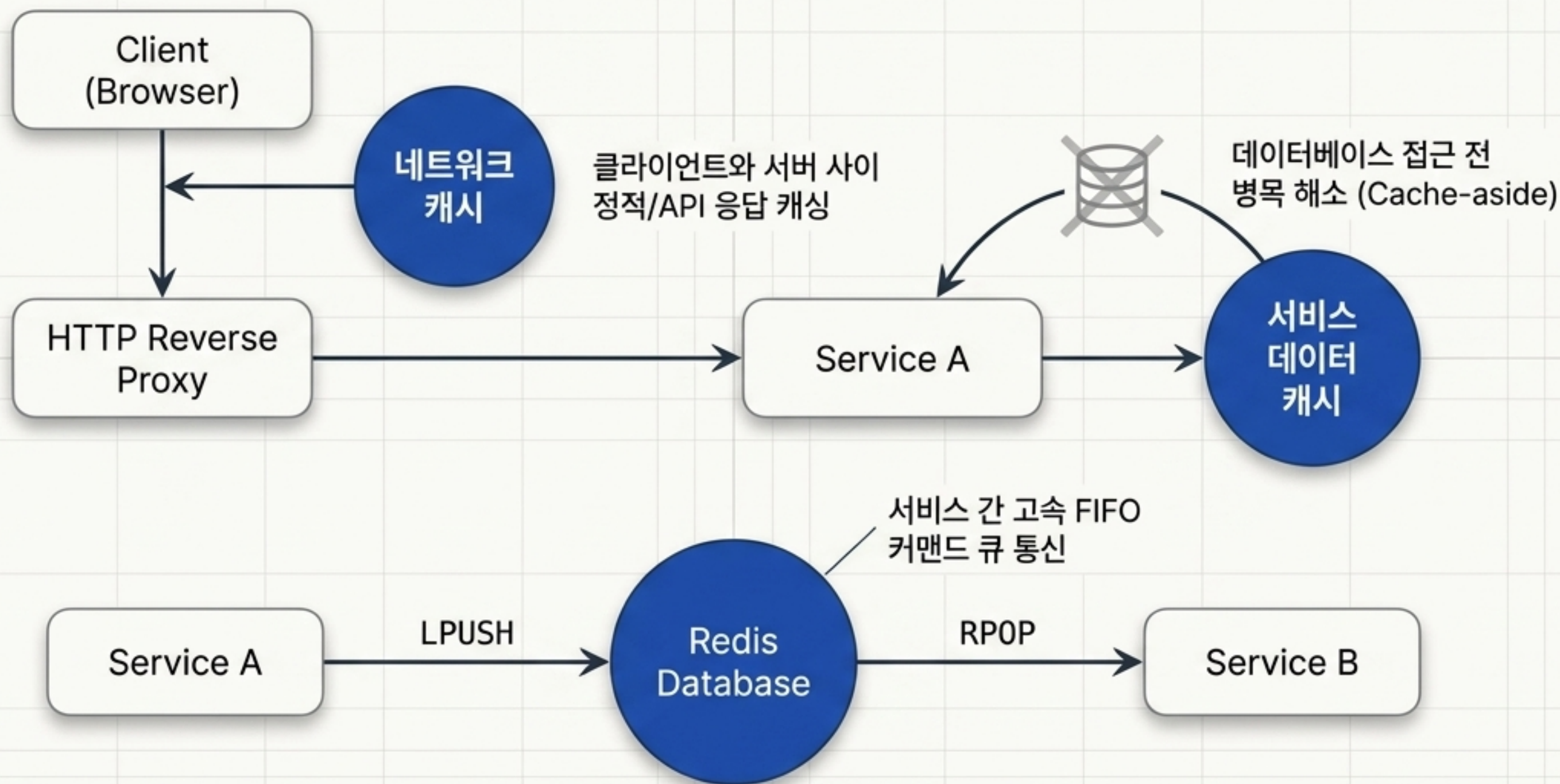
명령어: `volatile-lru`

휘발성을 넘어서: Redis 영속성(Persistence) 아키텍처



완벽한 엔터프라이즈 캐시는 AOF의 데이터 무결성과 RDB의 스냅샷 효율성을 결합하여 사용합니다.

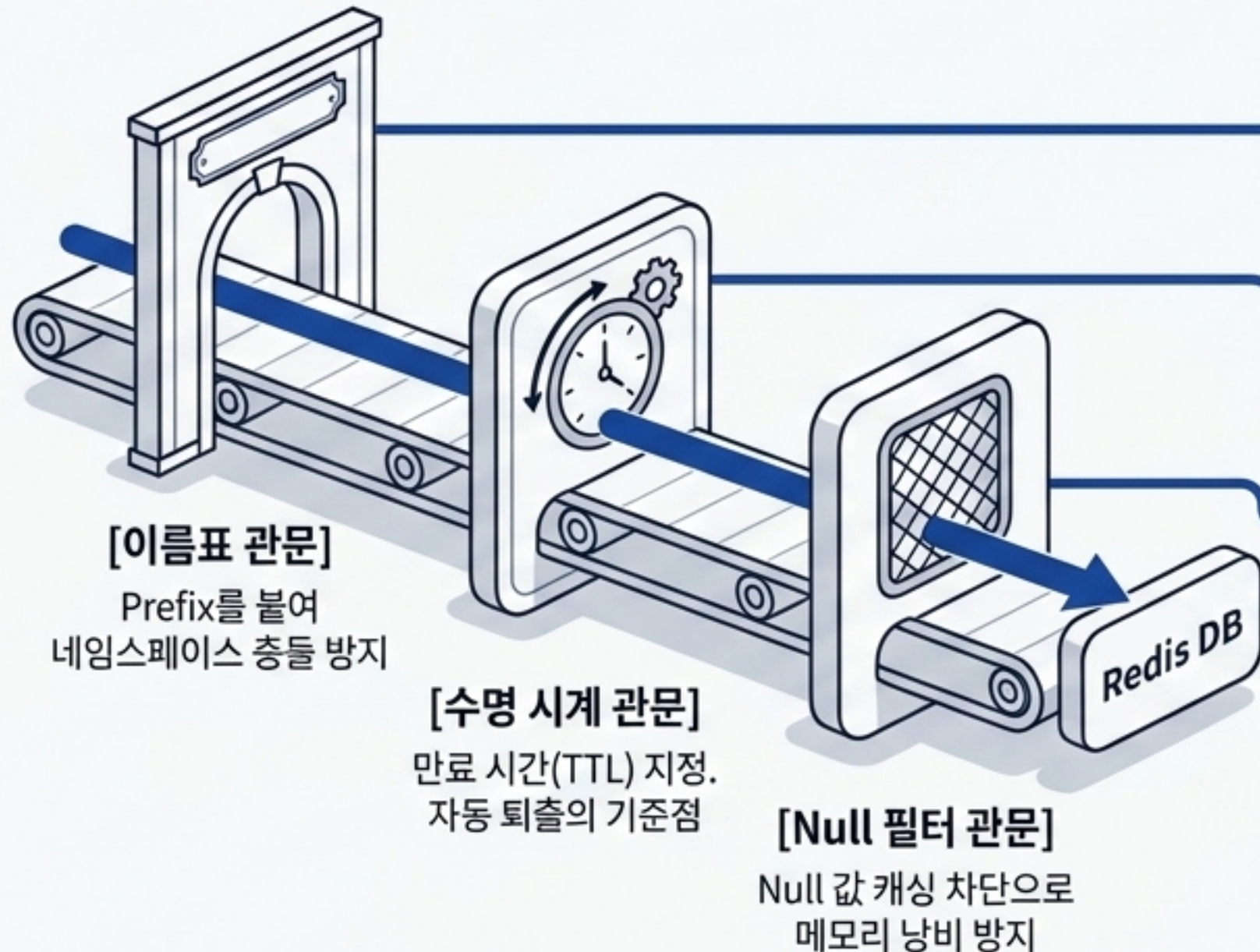
마스터 다이어그램: 마이크로서비스 생태계 속의 Redis



Redis는 단순한 캐시가 아닙니다. 현대 마이크로서비스 간의 통신과 데이터 흐름을 조율하는 초고속 메모리 백본(Backbone)입니다.

실무 구현: Spring Boot + Redis 파이프라인 구축

Graphic Pipeline

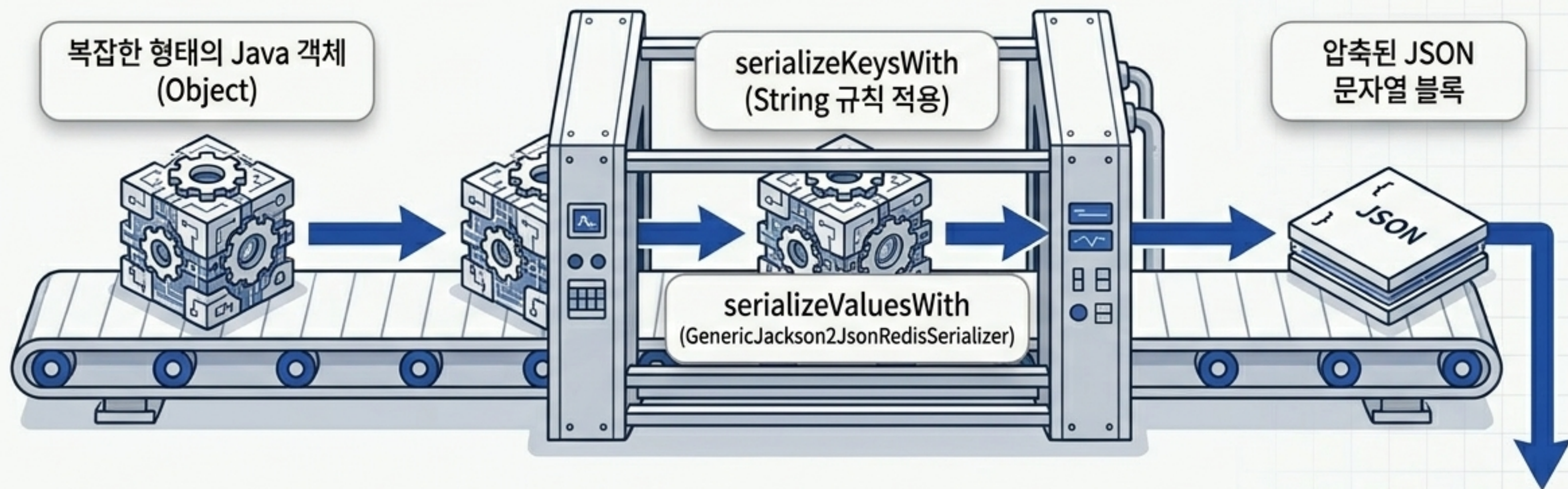


Code Mapping

spring-boot-starter-data-redis 의존성 추가로
ConcurrentMapCache를 RedisCacheManager로 전환.

```
1 @Configuration
2 @EnableCaching
3 public class RedisCacheConfiguration {
4
5     @Bean
6     public RedisCacheManager redisCacheManager(RedisConnectionFactory connectionFactory) {
7         RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
8             .computePrefixWith(cacheName -> cacheName + "::") // [이름표 관문]
9             .entryTtl(Duration.ofMinutes(60)) // [수명 시계 관문]
10            .disableCachingNullValues(); // [Null 필터 관문]
11
12         return RedisCacheManager.builder(connectionFactory)
13             .cacheDefaults(config)
14             .build();
15     }
16 }
```


데이터의 변환: 직렬화(Serialization) 컨베이어 벨트



⚠ List 역직렬화 에러 주의!

GenericJackson2JsonRedisSerializer 사용 시, 여러 개의 데이터를 한 번에 저장할 때 단순 List 타입 대신 명시적인 일급 컬렉션(First-Class Collection)을 선언하여 사용해야 타입 소거(Type Erasure)로 인한 런타임 역직렬화 에러를 완벽히 방지할 수 있습니다.

최종 검증: Redis 메모리 인스펙션

```
> keys *
```

```
1) "cache1::key_name"
```

Prefix가 적용된 캐시: 설정한 Prefix가 완벽하게 적용되어 다른 캐시 공간과 격리됨.

```
> get members
```

```
"{ \"users\": [ ... ] }"
```

일반 캐시: 기본 설정으로 저장된 일급 컬렉션 JSON 직렬화 데이터.

단순한 Key-Value 저장을 넘어, 정밀하게 조율된 캐싱 전략은 현대 엔터프라이즈 아키텍처의 필수 불가결한 생존 전략입니다.